Denote the nodes of the three by $V$. We will identify a sheep/shepherd with the node it occupies.

Notice that the task can be seen as an instance of the so called *set cover* problem: For every node $v \in V$, consider the set $S_v$ of sheep which would be protected if we put a shepher in node $v$. Then we need to find the smallest subset $P \subseteq V$ of nodes such that $\bigcup_{v \in P} S_v$ equals the set of all sheep. The general set cover problem is NP-complete, but the specific tree structure here will help us to solve the problem in polynomial time.

Let's start with the first subtask, the case when the tree is a chain. A shepherd can protect only the first sheep to the left and/or to the right, so the family $\{S_v \mid v \in V\}$ contains the following subsets:

- $\{x\}$ for every sheep $x$;
- $\{x, y\}$ for consecutive sheep $x, y$ such that $x - y$ is even.

We proceed with the following greedy algorithm: look at the first sheep. if the second one has the same parity, place a shepherd on the node halfway between them, otherwise place a shepherd in the same node as the first sheep. Next, remove the protected sheep and repeat the same algorithm. This solution has complexity $\mathcal{O}(N + K)$.

To solve the second subtask, we make use of representing the subsets of sheep by bitmasks in $\{0, 1, \ldots, 2^K - 1\}$, We first find the sets $S_v$ for every node $v \in V$ in $\mathcal{O}(K \cdot N)$ by running breadth-first search from every sheep.

We can then solve this set cover instance by dynamic programming. For each bitmask $mask$, let $f(mask)$ denote the minimum number of sets $S_v$ which cover $mask$. We iterate through the states $mask$ in increasing order. When calculating $f(mask)$, we iterate over all $submask \subseteq mask$, and eash time $submask$ corresponds to some of the sets $S_v$, we recalculate $f(mask) := f(mask \wedge submask) + 1$. To finish the transition, we put $f(submask) := f(mask)$ for all $submask \subseteq mask$,

Memory complexity is $\mathcal{O}(N + 2^K)$, and time complexity is $\mathcal{O}(K \cdot N + 3^K)$. (We leave the proof as exercise.)

For the remaining subtasks, we first pick an arbitrary node as the root. For each sheep $x$, we will refer to the set $\{v \in V \mid x \in S_v\}$ as its *territory*. We say that two sheep $x$ and $y$ are *friends* if their territories have nonempty intersection. The main idea is to greedily place a shepherd that protects some sheep and all of the (currently) unprotected friends of that sheep. The following claim will help us with that:

**Claim 1.** For some sheep $x$, let $a(x)$ be its highest ancestor that is in its territory. Then, $S_{a(x)}$ contains all friends of $x$ that are not deeper that $x$. *Proof.* Let $y$ be a friend of $x$ that is not deeper than $x$. If $y$ is in the subtree of $a(x)$, the claim is obvious, so assume the contrary. Take some node $z$ that is not part of territories of $x$ and $y$, and let $w$ be the midpoint of the path between $x$ and $y$. We have

$$d(z, x) = d(z, y) = d(z, w) + d(w, x) = d(z, w) + d(w, y),$$

where $d(u, v)$ denotes the distance between nodes $u$ and $v$. Also, for every sheep $t$ it's true that

$$d(z, w) + d(w, x) = d(z, x) \leq d(z, t) \leq d(z, w) + d(w, t),$$

which implies $d(w, x) \leq d(w, t)$, and $d(w, y) \leq d(w, t)$ is proved analogously. Hence, $w$ is contained in the intersection of territories of $x$ and $y$. Moreover, since $y$ is not deeper than $x$, $w$ is an ancestor of $x$, so it must be located on the path from $x$ to $a(x)$. Since $y$ is not contained in the subtree of $a(x)$, we have

$$d(a(x), y) \leq d(w, y) = d(w, x) \leq d(a(x), x),$$

so a shepherd in $a(x)$ protects $y$, as needed. $\square$

According to Claim 1, the following algorithm is correct:

- Repeat until all sheep are protected:
  - Place a shepherd in $a(x)$, where $x$ is (one of) the deepest currently unprotected sheep.

The straighforward implementation in the complexity $\mathcal{O}(N(N+K))$, is sufficient to solve the third subtask. To solve the fourth subtask, we need to speed up the algorithm. For each node $v$, let $dep(v)$ and $dist(v)$ denote the depth of the node and the distance to the closest sheep, respectively. We can calculate $dist$ by running a BFS starting from each sheep. Alternatively, we can imagine that we added a *dummy* node connected to all the sheep and run the BFS starting in that node. The following fact will help us to efficiently determine $a(x)$ for each sheep $x$:

**Observation 2.** If $v$ is an ancestor of $x$, then $dist(v) \leq dep(x) - dep(v)$, where equality is attained if and only if $v$ is in the territory of $x$.

Due to the above, $a(x)$ is the index of first maximum of $dist(v) + dep(v)$ over all $v$ on the path from the root to $x$. Thus we can calculate $a(x)$ for each sheep $x$ by a simple depth-first search from the root.

The only thing left is maintaining the deepest unprotected sheep efficiently.

If we sort the sheep by decreasing depth, we can reduce this problem to maintaining the set of protected sheep.

Consider the directed graph $G$ with the same vertex set $V$ as the given tree, and with edges $(u, v)$, where $\{u, v\}$ is an edge from the given tree such that $dist(v) = dist(u) + 1$.

**Observation 3.** For each node $v \in V$, $S_v$ is exactly the set of sheep $x$ such that there exists a path in $G$ from $x$ to $v$.

Whenever we place a new shepherd, we can run a DFS from that node following edges of $G$ backwards, ignoring the nodes that were alredy visited. According to Observation 3, sheep is protected if and only if it was visited by the DFS. Since each node is visited at most once, the complexity of this part is linear.

Complexity of the solution is $\mathcal{O}(N + K \log K)$. Notice that the factor $\log K$ comes from sorting the sheep by depth, which is of course possible to do with complexity $\mathcal{O}(N + K)$ because the depths are at most $N$. Therefore it's possible to solve the task in linear complexity. For implementation details, see the official source codes.